

Tour Ceylon

Travel Component Demonstration

Screen script · File connections · Panel answers

Project ID	R26-IT-152
Student	Warniarachchi P.M.R. (IT22225092)
Component	AI Itinerary Generator (Travel)
Date	13 August 2026
Travel URL	http://localhost:5180
API URL	http://127.0.0.1:5002/api

How to use this PDF while presenting

For each screen: read the “Say this” line in simple English, click what “Do this” says, and if they ask “which file?”, use the Files + How connected lines.

1. Big picture (30 seconds)

Travel is the trip planner. A tourist logs in, picks places, sets money, chooses hotels, gets a day-by-day plan with times, finds local help, and downloads a PDF. History opens old trips. SOS opens from the same login.

Path on screen: Login → Attractions → Budget → Stay → Itinerary → Discover → Export

Path in code: React page → api/client.js → Flask route → service → PostgreSQL table

One sentence if they ask “what did you build?”

I built a React + Flask trip wizard that turns moods, attractions, budget, and hotels into a rule-based timed itinerary, discovers local services, and exports a PDF.

2. Tech (short answers)

Frontend?	React 18 + Vite — folder frontend/travel — port 5180
Backend?	Flask API — backend/run.py — port 5002
Database?	PostgreSQL — tables for users, trips, hotels, itineraries
Login?	JWT token saved in browser localStorage
PDF?	ReportLab in pdf_service.py
AI / ML?	No OpenAI / no Random Forest. Times from rules in ai_service.py
Charts?	Recharts pie chart on Budget page

3. How files connect (whole system)

Simple chain for every click:

- **Page** (what user sees) — e.g. ItineraryPage.jsx
- **Context** — AuthContext (login) + TripContext (current trip choices)
- **API client** — api/client.js sends request + JWT
- **Route** — e.g. routes/itinerary.py receives URL
- **Service** — rules (trip_plan_service / ai_service / budget_service / pdf_service)
- **Model / DB** — SQLAlchemy model ↔ PostgreSQL table

frontend/travel/src/pages/	Each wizard screen
frontend/travel/src/context/	Shared login + trip memory
frontend/travel/src/api/	Talks to Flask
frontend/travel/src/components/	Sidebar Layout, photos
backend/app/routes/	API URLs
backend/app/services/	Business rules
backend/app/models/	Database tables
backend/data/travel/	Excel/CSV seed data

4. Screen-by-screen demo script

Start both servers first: backend python run.py · frontend npm run dev · open :5180

Screen A — Login

Say this: This is the login screen. Users create an account or sign in. The server checks the users table and gives a JWT token. The browser keeps that token for later API calls.

Do this: Register or login with a demo account. Show the app moves to Attractions.

Frontend	LoginPage.jsx · AuthContext.jsx · api/client.js
Backend	routes/auth.py · models/user.py
Saved	users table · localStorage access_token

How connected: LoginPage → AuthContext.login() → client.js POST /api/auth/login → auth.py → users table → JWT back to browser.

Screen B — Attractions (step 1)

Say this: Here the tourist sets trip days, picks moods like Adventure or Spiritual, and selects attractions. Moods filter the list from the database. When they confirm, we create the trip record.

Do this: Set days (e.g. 3). Pick 1–3 moods. Select a few places. Click confirm / continue.

Frontend	AttractionsPage.jsx · AttractionPhoto.jsx · TripContext · client.js
Backend	routes/attractions.py · routes/trip_input.py · models/attraction.py · user_trip_input.py
Saved	user_trip_input (days, moods, attraction IDs)

How connected: AttractionsPage loads GET /attractions?moods=... → attractions.py filters SQL by mood_tag. Confirm → POST /trip-input → trip_input.py saves user_trip_input.

Screen C — Budget (step 2)

Say this: Now we set money. The user enters a total budget. The system splits it into food, stay, shopping, and transport. Default is about 25% / 35% / 15% / 25%. The pie chart shows that split.

Do this: Enter a total (e.g. 150000). Show the pie chart. Save / continue.

Frontend	BudgetPage.jsx · Recharts · TripContext · budgetApi in client.js
Backend	routes/budget.py · services/budget_service.py · models/budget_split.py
Saved	user_trip_input.budget + budget_split row

How connected: BudgetPage → POST /budget/split → budget.py → budget_service.py does the math → budget_split table.

Screen D — Stay / Accommodation (step 3)

Say this: For each city in the trip, the tourist picks a hotel. Hotels are filtered by the stay budget and matched to the destination name from our hotel catalog.

Do this: Pick one hotel per destination. Continue.

Frontend	AccommodationPage.jsx · accommodationApi · tripApi.update
Backend	routes/accommodation.py · trip_plan_service.py · budget_service.py · models/accommodation.py
Saved	user_trip_input.accommodations JSON

How connected: AccommodationPage → GET /accommodation?trip_id= → accommodation.py uses trip destinations + stay budget → hotels listed → PATCH trip-input saves chosen hotels.

Screen E — Itinerary (step 4) ★ main feature

Say this: *This is the main part. The system builds a day-by-day plan with start times. It is rule-based: first split days by destination, then set times using moods and categories — for example heritage earlier, beach later. We skip lunch time and finish around 8 PM. Important: this is NOT ChatGPT and NOT machine learning.*

Do this: Click Generate (or Rebuild). Scroll days. Point at times and destinations.

Frontend	ItineraryPage.jsx · itineraryApi in client.js
Backend	routes/itinerary.py · trip_plan_service.py · ai_service.py · models/generated_itinerary.py
Saved	generated_itinerary.itinerary JSON

How connected: *ItineraryPage → POST /itinerary/generate → itinerary.py → trip_plan_service splits days → ai_service adds times → save JSON in generated_itinerary → page shows result.*

Screen F — Discover / Recommendations (step 5)

Say this: *After the plan, tourists can find local businesses, travel agencies, and licensed guides near their trip cities, and save favorites to the trip.*

Do this: Open Businesses / Agencies / Guides. Save one item.

Frontend	RecommendationsPage.jsx · recommendationsApi · savedRefsApi
Backend	routes/recommendations.py · routes/saved_references.py · business/agency/guide models
Saved	saved_references table

How connected: *Page calls GET with trip_id → recommendations.py filters by destination (SQL ILIKE) → user saves → POST saved-references.*

Screen G — Export (step 6)

Say this: Last step. We show a summary of the trip and download a PDF made from the saved itinerary. Nothing new is invented here — it prints what we already generated.

Do this: Click download PDF. Open the PDF briefly for the panel.

Frontend	ExportPage.jsx · itineraryApi.pdfUrl + token fetch
Backend	routes/itinerary.py (PDF route) · services/pdf_service.py
Saved	No new row — PDF built from existing trip + itinerary

How connected: ExportPage → GET /itinerary/<trip_id>/pdf with JWT → itinerary.py → pdf_service.py → PDF bytes.

Screen H — History (side path)

Say this: History lists past trips. Opening one puts the old trip back into memory so we can view or export again. This proves data is saved in the database, not only in the browser.

Do this: Open History. Click a past trip. Show it loads on Itinerary.

Frontend	HistoryPage.jsx · TripContext reload
Backend	itinerary history + trip-input GET + budget GET
Saved	Reads existing user_trip_input + generated_itinerary

How connected: HistoryPage loads past rows → user picks one → frontend refills TripContext from API → go to Itinerary.

Screen I — Profile + SOS button

Say this: Profile holds visitor details used with SOS, like hotel and emergency contact. The SOS button opens the Tourist SOS app using the same Travel login — no need to register again.

Do this: Optional: open Profile, then click SOS in the top bar.

Frontend	ProfilePage.jsx · Layout.jsx · api/sosClient.js · theme/sosProfile.js
Backend	routes/tourists.py (bridge-travel + profile)
Saved	Tourist SOS profile / session token

How connected: SOS click → sosClient.js POST /api/tourists/bridge-travel with Travel JWT → SOS session → open tourist app.

5. Full file map (if they ask “show me the files”)

5.1 Frontend files

main.jsx	Starts app; wraps Auth + Trip providers
App.jsx	Routes; blocks pages until login
api/client.js	All Travel API calls + JWT header
api/sosClient.js	Bridge / open Tourist SOS
context/AuthContext.jsx	Login state + token
context/TripContext.jsx	Current trip wizard state
components/Layout.jsx	Sidebar steps, progress, SOS / History / Profile
components/AttractionPhoto.jsx	Place photos
components/AttractionDetailModal.jsx	Attraction details popup
pages/LoginPage.jsx	Register + login
pages/AttractionsPage.jsx	Days, moods, places
pages/BudgetPage.jsx	Money + pie chart
pages/AccommodationPage.jsx	Hotels
pages/ItineraryPage.jsx	Day plan
pages/RecommendationsPage.jsx	Discover + save
pages/ExportPage.jsx	PDF download
pages/HistoryPage.jsx	Past trips
pages/ProfilePage.jsx	Visitor / emergency profile

5.2 Backend routes

routes/auth.py	Register / login / me
routes/attractions.py	List attractions by mood
routes/trip_input.py	Create / update trip
routes/budget.py	Budget split save/get
routes/accommodation.py	Hotels for trip cities
routes/itinerary.py	Generate / get / history / PDF
routes/recommendations.py	Businesses, agencies, guides
routes/saved_references.py	Saved favorites
routes/tourists.py	SOS bridge + profile

5.3 Backend services + models

services/budget_service.py	Budget % and money math
services/trip_plan_service.py	Split days; match hotels to cities
services/ai_service.py	Activity times from rules (no OpenAI)
services/pdf_service.py	Build itinerary PDF
models/user.py	Travel accounts
models/attraction.py	130 attractions
models/accommodation.py	944 hotels
models/user_trip_input.py	Trip core row
models/budget_split.py	Budget categories
models/generated_itinerary.py	Saved day plan JSON
models/business_directory.py / travel_agency.py / tourist_guide.py	Discover catalogs
models/saved_reference.py	Favorites on a trip

6. How itinerary timing works (simple)

Day split	One day per city first; extra days reuse places	trip_plan_service.py
Start time	From selected moods (or user picks a time)	ai_service.py
Category bias	Wild / heritage earlier; beach / spa later	ai_service.py
Name hints	Words like temple / beach / safari adjust time	ai_service.py
Spacing	About 2–3 hours between activities; skip 12–1 lunch	ai_service.py
End of day	Stop around 8 PM	ai_service.py

7. Data counts (seeded)

Attractions	130	attractions.xlsx
Hotels	944	accomadation.xlsx
Businesses	126	business_directory.xlsx
Agencies	417	travel_agencies.xlsx
Guides	2,906	tourist_guides.csv

8. Questions they may ask — short answers

Is this AI / ML?	No ML model. Rule-based planner. File <code>ai_service.py</code> only sets times from rules.
Why name <code>ai_service</code> ?	Legacy name. Logic is lookups + heuristics, not OpenAI.
How hotels match?	Stay budget filter + destination name/address token match.
How attractions filter?	SQL WHERE <code>mood_tag</code> IN selected moods.
Where is trip saved?	PostgreSQL <code>user_trip_input</code> + <code>generated_itinerary</code> .
What if I refresh?	TripContext memory clears, but saved trip reloads from History / DB.
How PDF is made?	<code>pdf_service.py</code> reads saved itinerary and builds PDF with ReportLab.
How SOS connects?	bridge-travel exchanges Travel JWT for SOS tourist session.
Frontend port?	5180
Backend port?	5002
Auth method?	Flask-JWT-Extended; Bearer token on protected APIs.
Any training data / accuracy?	No model training. Quality checked manually + API smoke tests.
Budget split default?	About 25% food, 35% stay, 15% shopping, 25% transport.
Can user change times?	Yes — Auto start or fixed time, then Rebuild on Itinerary page.
Discover data real?	Yes — seeded from Excel/CSV catalogs (agencies, guides registry).

9. How to run before demo

- Terminal 1: cd backend → activate .venv → python run.py
- Terminal 2: cd frontend → npm run dev (or npm run dev:travel)
- Open <http://localhost:5180>
- If empty data: run `apply_schema.py` then `seed_data.py` under `backend/scripts/travel/`
- Health check: open <http://127.0.0.1:5002/api/health>

10. Closing line

Say at the end

So overall: each screen is a React page, each action goes through `client.js` to a Flask route, services apply the rules, and PostgreSQL keeps the trip — then Export turns that saved plan into a PDF.

Project R26-IT-152 · Warniarachchi P.M.R. (IT22225092) · AI Itinerary Generator · 13 August 2026